

Cache-Aware Client-Side Request Planning for Black-Box LLM APIs

Prajjwal Chittori

Independent Researcher

prajjwalchittoriwork@gmail.com

ORCID: [0009-0000-7410-8290](https://orcid.org/0009-0000-7410-8290)

July 2026

Abstract

Most applications consume large language models through black-box, per-token-billed APIs. The developer does not own the model weights, cannot change server kernels, and is billed for every token the server processes. A common folk belief holds that this bill can be lowered by relocating work to the client, for example by tokenizing on the device and shipping token identifiers, or by prompting in a more information-dense language. We show that the relevant invariant rules these out: a consumer pays for the tokens the server processes, so moving computation to the client changes the bill only if it reduces billable tokens or calls at fixed task quality. Under that constraint the space of useful client-side interventions is small and well defined. We give a cost model for it, a taxonomy that separates the few legal moves from the many that are null or impossible, and we identify one intervention that is simultaneously lossless (no task-quality cost) and local-compute-free: aligning requests to the provider's own prompt cache. We present a greedy prefix-clustering scheduler that reorders and times a request stream to maximize provider-cache hits under a short time-to-live, and we characterize the regime in which it helps. In simulation under representative public cache parameters, the scheduler reduces billed cost by up to 60 percent on an agentic tool-use workload with no quality loss, dominating prompt compression and semantic caching on cost, local compute, and quality at the same time. We also show where it stops helping: when the request rate is so slow that even back-to-back same-prefix calls fall outside the cache window, and on human-paced chat where within-session gaps are not reorderable.

1. Introduction

Building on a paid LLM API is now the default. The economic unit is the token: providers bill for input tokens and output tokens, and increasingly discount input tokens that hit a server-side prompt cache. Application developers in turn spend significant effort trying to lower that bill.

Many of the proposed tricks do not survive scrutiny. Tokenizing on the client and sending token identifiers saves nothing, because the server processes the same number of tokens regardless of where the text was split, and the forward pass, not the wire transfer, is what costs money and latency. Prompting in a denser language can shift the token count, but the effect is a property of the tokenizer's training corpus rather than the language, and any gain is typically paid back in degraded reasoning quality. The large literature on edge-cloud collaborative inference and split execution does reduce real compute, but it assumes the consumer owns the model weights, which the API consumer does not.

The starting point of this paper is the invariant that governs the black-box, per-token setting:

A consumer is billed for the tokens the server processes. Relocating computation to the client changes the bill only by reducing billable tokens or the number of calls, at fixed task quality.

This single constraint collapses the design space. It is what makes client tokenization null and split execution impossible. It also exposes a smaller set of moves that are genuinely available, and lets us reason about each one in the same currency. Among them, one stands out: aligning a request stream to the provider's prompt cache is lossless (the tokens are identical, only their arrangement changes) and costs essentially no local compute. We argue this is the move an API consumer should reach for first, and we build the planner that does it.

Contributions:

1. A cost model for the black-box API consumer that expresses billed cost as a function of full and cache-discounted input tokens, output tokens, and call count, and treats each client-side intervention as a transform on that function with an attached local-compute price (Section 2). This yields a per-intervention break-even condition, and a negative result: an explicit account of which moves are null or impossible in this setting.
2. A characterization and exploitation method for the provider's opaque prompt cache, recovered from usage telemetry, and a greedy prefix-clustering scheduler that maximizes cache hits under a short time-to-live (Sections 3 and 4).
3. An open-source, tokenizer-agnostic artifact and a simulation study that quantifies when shaping wins and when it does not (Sections 5 and 6).

2. Setting and Cost Model

2.1 The consumer's view

We model a single tenant calling one provider. Each request carries a prefix (the stable part: system prompt, tool schemas, few-shot examples, large documents) and a suffix (the variable part: the user turn). The provider may cache a prefix so that a later request sharing that exact prefix pays a discounted rate on the cached portion. The consumer cannot build or inspect this cache directly. The consumer can, however, observe its effect: modern APIs report the number of cached input tokens in the usage payload of each response [7, 8].

2.2 Cost model

Let a request have `p` prefix tokens and `s` suffix tokens, and produce `o` output tokens. Let the provider charge `c_in` per input token and `c_out` per output token, with a cache read priced at a fraction `r` of `c_in` and a cache write at a multiple `w` of `c_in`. For a single request, given that `k` of its prefix tokens were served from cache (a read) and the remaining prefix was either written to the cache (a write) or paid at full rate, the billed cost is:

$$\text{cost} = c_in * (k*r + \text{written}*w + \text{full}) + c_in * s + c_out * o$$

where `k + written + full = p`. The suffix and the output are always at full rate. This is the function our artifact implements exactly (`ccrp.cost_model.billed_cost`).

Each client-side intervention is a transform on this function plus a local-compute term. An intervention is worth applying when the server dollars it saves exceed its local cost:

$$\text{net_savings} = \text{baseline_cost} - \text{intervention_cost} - \text{local_cost} > 0$$

Cache-aware shaping is the unique intervention whose `local_cost` is approximately zero and whose task-quality cost is exactly zero, because it does not alter, drop, or summarize any tokens. It only changes the order

and timing of requests so that more of each prefix is served as a cache read. Its benefit is therefore bounded only by the cacheable fraction of the workload, which is what the rest of the paper measures.

3. The Consumer Design Space

We enumerate the moves available to a black-box API consumer and label each. The negative entries are part of the contribution: they explain why the consumer's design space is so much smaller than the model-owner's.

Move	Status	Reason
Client tokenization, ship token ids	Null	Billing and forward-pass compute are invariant to where tokenization runs
Speculative decoding with client draft	Impossible	No server verification interface, no weights
Early-layer or split execution	Impossible	No model weights
Output constrained decoding	Server-side	Decoding happens on the provider
Prompt or context compression	Legal, lossy, local cost	A baseline (Section 6)
Semantic caching of responses	Legal, skips calls on a hit	A baseline (Section 6)
Model cascade (FrugalGPT [1])	Legal, routing across models	One cell of this taxonomy
Cache-aware request shaping	Legal, lossless, free	The centerpiece (Section 4)

The distinction from the server-side literature is sharp. Systems such as PagedAttention in vLLM [2], RadixAttention in SGLang [3], and Mooncake [4] make prefix reuse cheap by managing the key-value cache on hardware the operator controls. The black-box consumer has none of that control. The consumer's only lever on the provider's cache is the content and timing of the requests it sends, which is exactly what shaping manipulates.

4. Cache-Aware Request Shaping

Shaping is a family of techniques, not one trick. They share a precondition and culminate in a scheduler.

4.1 Static discipline

Two layout rules are necessary for any cache hit at all. First, place invariant content (system prompt, tool schemas, few-shot, documents) at the front and variable content (the user turn) at the tail, so the longest possible prefix is cacheable. A request that places a timestamp or a session identifier at the top busts the entire prefix on every call. Second, canonicalize the prefix so it is byte-identical across requests: sort object keys (nondeterministic key ordering in tool schemas is a common silent cache-buster) and normalize incidental whitespace. Our artifact provides `canonical_prefix`, which produces a stable key for a prefix built from text and structured parts regardless of key order or whitespace.

4.2 The scheduling problem

Provider caches expire after a short time-to-live (on the order of minutes). Given a stream of requests with shared prefixes, the consumer can often choose the order and timing of dispatch within a latency-slack budget. Two same-prefix requests that the naive arrival order separates by more than the time-to-live each pay a cache write; the same two, scheduled back to back, pay one write and one read. This is the client-side analogue of

what RadixAttention [3] does inside the server, except the consumer is optimizing against a cache it cannot see and does not own.

4.3 A greedy scheduler

We deliberately use the simplest scheduler that captures the effect. `greedy_schedule` maintains a model of the cache (a prefix key mapped to an estimated warm-until time) and, at each step among the requests that have arrived, applies a four-way priority:

1. Deadline guard: any request at or past its slack deadline (`arrival + max_slack`) is served first, so reordering never starves a request.
2. Otherwise, prefer a request whose prefix is currently warm.
3. Otherwise, prefer a request whose prefix matches the one just served, to begin a warm streak.
4. Otherwise, take the earliest arrival.

The contribution is the claim that this trivial policy captures most of the available benefit, not that it is optimal. The schedule it returns is a permutation of the input requests: no request is dropped or altered, which is what makes the intervention lossless by construction.

5. Implementation

The artifact (`ccrp`, roughly 350 lines of Python plus tests) is tokenizer-agnostic: it works in token counts, so the simulator and scheduler need no live model. Modules: `cost_model` and `cache_sim` (the pricing and cache model), `canonicalize` and `clustering` (key derivation and grouping), `simulate` (pricing a fixed order) and `scheduler` (the greedy planner), `pipeline` (the end-to-end canonicalize, cluster, schedule, price chain), `workloads`, `baselines`, `eval`, `metrics`, and `figures`. A separate `characterize` module recovers cache parameters from real provider telemetry, with the provider SDKs imported only behind a main guard so the core has no network dependency. The suite has 34 tests.

6. Evaluation

6.1 Methodology

We use a hybrid method. The cache parameters (time-to-live, minimum cacheable prefix length, read discount, write multiplier) are recovered for real providers from the usage telemetry that each response reports, using small probe sequences. The scheduler is then evaluated against a calibrated simulator, which lets us sweep many orderings and operating points without a large API bill. The results below use representative public parameters in the style of current providers (time-to-live 300 seconds, minimum prefix 1024 tokens, read at 0.1x, write at 1.25x, input at 3.0 and output at 15.0 dollars per million tokens). Absolute dollar amounts scale with workload size and are illustrative; the reductions are the result.

Two workloads, chosen for prefix structure: agentic tool-use (a large shared system prompt and tool schemas reused across every step of every run) as the primary, and multi-turn chat (a shared, growing per-session prefix under user think-time) as the secondary.

6.2 Agentic tool-use: a clean regime

We sweep the per-step service time, which stands in for how long each model call and its surrounding agent step takes. With four interleaved runs, the naive arrival order separates one run's consecutive steps by four service times, while the shaped order keeps them adjacent.

service time (s)	naive (\$)	shaped (\$)	reduction
10	82.20	82.20	0%
40	82.20	82.20	0%
75	206.40	82.20	60.2%
100	206.40	82.20	60.2%
150	206.40	82.20	60.2%
300	206.40	206.40	0%

The structure is the finding. Below a threshold (here 75 seconds, the point at which the interleaved gap of four service times reaches the 300 second time-to-live) the naive order already keeps every prefix warm, so shaping has nothing to add. Above that threshold, the naive order re-writes each run's prefix on every step while the shaped order keeps it warm, for a 60 percent billed-cost reduction at zero quality cost. Above a second threshold (300 seconds, where even adjacent same-prefix calls fall outside the window) the cache is useless at any ordering and shaping again does nothing. Shaping is valuable precisely in the band where the request rate is fast enough for grouped reuse to land inside the cache window but the interleaved order does not.

6.3 Multi-turn chat: a harder case

For chat we sweep user think-time between turns, with six interleaved sessions.

think-time (s)	naive (\$)	shaped (\$)	reduction
10	156.15	156.15	0%
60	156.15	156.15	0%
120	156.15	156.15	0%
300	321.75	240.33	25.3%
600	321.75	321.75	0%

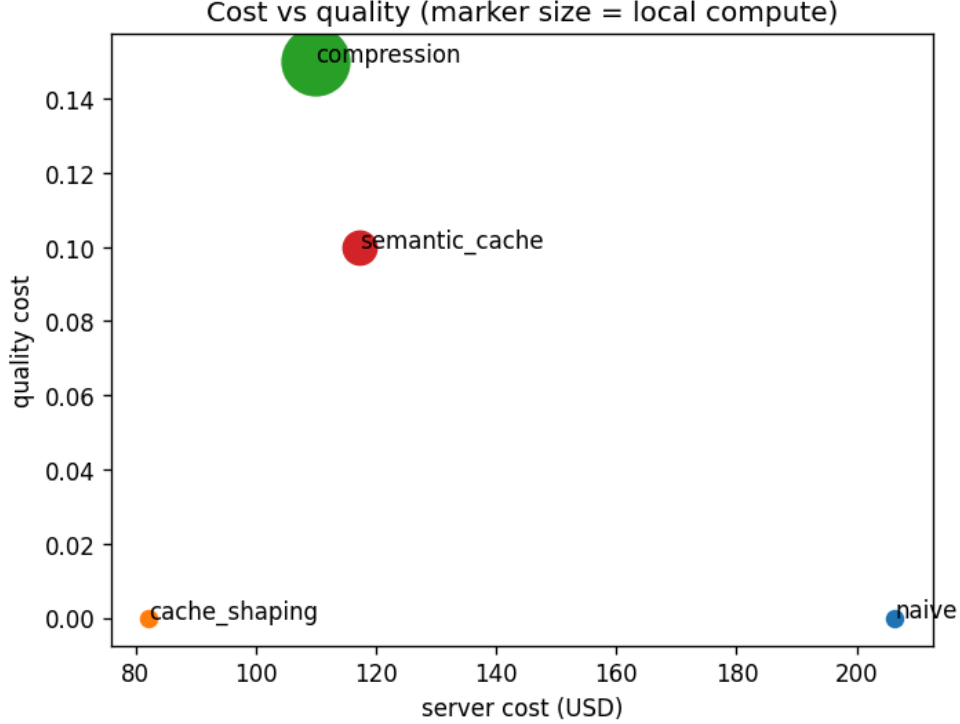
Chat is less favorable, and honestly so. While think-time stays within the cache window, the naive order already holds the cache. Once think-time exceeds the window, shaping can recover only the cross-session cache pressure, not the within-session gaps: the scheduler cannot make a user reply faster, so a single session's consecutive turns remain separated by the think-time no matter how requests are ordered. The 25 percent recovery at the 300 second point, and the absence of any recovery at 600 seconds, reflect this limit. Shaping helps machine-paced, reorderable workloads more than human-paced ones. (The chat numbers also use a v1 approximation in which a session's growing prefix is treated as one refreshing key; we note this as a limitation.)

6.4 The comparison plane

At the agentic operating point (service time 100 seconds) we place shaping beside the two baselines and the naive order, on three axes: server dollars, local-compute dollars, and a quality cost.

method	server (\$)	local (\$)	quality
cache_shaping	82.20	0.000	0.00
compression	109.92	0.160	0.15
semantic_cache	117.15	0.032	0.10
naive	206.40	0.000	0.00

In this regime shaping Pareto-dominates: it has the lowest server cost and the only intervention with zero local compute and zero quality cost. Compression and semantic caching both beat the naive order, but each pays a quality and a local-compute price and still costs more than shaping. This is the thesis made concrete: where the cacheable prefix fraction is high, the lossless and free intervention is also the cheapest, and the lossy ones are a fallback for the regimes where shaping cannot help. Figure 1 plots these points.



7. Related Work

FrugalGPT [1] reduces cost for API consumers by cascading from cheap to expensive models. It is the closest prior work and occupies one cell of our taxonomy (model routing); it does not formalize the broader client-side space or the cache-alignment move. Prompt and context compression (LLMLingua [5] and its variants) reduces input tokens by dropping low-information content; it is one of our baselines and is lossy. Semantic response caching (for example GPTCache [6]) skips calls on a near-duplicate query; it is our other baseline and depends on a hit. Server-side prefix reuse (PagedAttention in vLLM [2], RadixAttention in SGLang [3], Mooncake [4]) achieves the same economic effect as shaping but requires owning the serving stack; our contribution is to obtain a share of that benefit from the outside, against an opaque provider cache. Provider prompt caching itself (introduced across major APIs in 2024 [7, 8, 9]) is the mechanism we exploit; to our knowledge the client-side problem of scheduling against it has not been studied.

8. Limitations

The provider cache is opaque and may change between characterization and deployment; we report the characterization method and date so results are reproducible against telemetry rather than against undocumented internals. Our evaluation is simulation under representative parameters, not an end-to-end live-API study; the artifact supports the live characterization that would calibrate it. The chat model uses a v1 approximation of growing prefixes. The scheduler is greedy, not optimal, by design. We evaluate a single tenant and do not model provider-side cache capacity eviction, only time-to-live.

9. Conclusion

For a consumer of a black-box, per-token LLM API, the bill is set by tokens the server processes, and the only honest way to lower it is to send fewer billable tokens or calls at fixed quality. Within that constraint, aligning requests to the provider's prompt cache is the rare intervention that costs nothing and loses nothing, and a trivial greedy scheduler captures most of its benefit in the regime where it applies. We have framed that regime, built the planner, and shown both where it wins and where the consumer must fall back to lossy compression or caching. The framing and the artifact are meant to be reused: the cost model and the taxonomy outlive any single provider's pricing, and the characterization harness adapts as those prices move.

References

- [1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2305.05176.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.
- [3] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.
- [4] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025. Best Paper Award. arXiv:2407.00079.
- [5] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358-13376, 2023. arXiv:2310.05736.
- [6] Fu Bang. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, pages 212-218, 2023.
- [7] Anthropic. Prompt Caching. Anthropic API technical documentation, 2024.
- [8] OpenAI. Prompt Caching. OpenAI API technical documentation, 2024.
- [9] Google. Context Caching. Gemini API technical documentation, 2024.